

Restoring Crunchyroll Playback in Helium on macOS

A reproducible Widevine and DNS case study

Prepared by Codex from a user-authorized local troubleshooting session October 2, 2026 | Apple silicon macOS | Technical report, revision 3

Abstract

Crunchyroll failed to play an episode in Helium, displaying KAT-6005. Investigation found an initially absent Widevine content decryption module, a blocked player-service connection, and a DRM initialization failure that persisted after installing two Google-signed Widevine versions. Changing the browser's secure DNS provider to Cloudflare resolved the observed connection refusal but did not restore playback. A separate, locally re-signed Helium copy with a macOS library-validation exception and bundled Widevine successfully played an independent protected-video test and the Crunchyroll episode. The final patch was then applied to the main installation with normal Keychain access and a complete app/profile backup. Its existing Crunchyroll login remained usable, and playback advanced from 11:20 to 11:28 with visible video and an audio-playing indication.

This is a successful local workaround, not official Helium DRM support. The experiment supports library validation as a likely blocker, but the successful intervention changed both signing configuration and module placement. The user subsequently identified their internet connection as the cause of the observed X loading problem. It is therefore no longer classified as a suspected DRM-patch regression. Future updates, passkeys, other streaming services, and longer playback sessions were not validated.

1. Scope and environment

Item	Observed configuration
Machine	Apple silicon, arm64
Operating system	macOS 27.2 beta, as observed during the session
Initial Helium	0.18.1.1; Chromium 154.0.8037.57
Helium used for the successful patch	0.18.2.1; Chromium 154.0.8037.92
Chrome supplying the final CDM	Framework version 153.0.8010.53
Widevine versions tested	4.10.3112.0 and 4.10.3050.0, arm64
Final Widevine version	4.10.3112.0
Main app	/Applications/Helium.app
User data	~/Library/Application Support/net.imput.helium

Helium updated during the first restart, so the initial and successful trials were not performed on an identical browser version. The target was The Misfit of Demon King Academy, season 2 episode 5, "In Between Royalty and Mixed Blood," with a displayed duration of 23:40. The test account already had access to the episode. No account, region, or subscription controls were changed.

Helium's public DRM feature request provided context [1], and its Widevine installer change provided a relevant implementation reference [2]. Neither reference establishes that this local configuration is officially supported.

2. Diagnosis: registration was not initialization

The original Widevine directory existed but was empty. A copy of Chrome's Google-signed Widevine 4.10.3112.0 was placed in Helium's user-data component directory. File hashes matched the source, and the module's macOS signature verified. After restart, `chrome://components` listed the version and `chrome://media-internals` listed the CDM as enabled. Crunchyroll nevertheless continued to show KAT-6005. A registered module therefore did not prove that its library could initialize.

2.1 A separate network failure

The browser console reported `ERR_CONNECTION_REFUSED` for `licensing.bitmovin.com/licensing`, among other services. Direct DNS queries to the router returned `0.0.0.0` and `::` for the hostname. A direct query to `1.1.1.1` also returned a blocked answer, while HTTPS DNS queries to Cloudflare and Google returned a routable address. System resolution and IPv4 HTTP access differed from the direct-query result.

These observations were consistent with DNS filtering or interception, but did not identify its owner or exact mechanism. With the user's explicit approval, Helium's secure DNS provider was changed from the network default to Cloudflare. The endpoint then loaded without a browser connection error. Crunchyroll still failed. The DNS change resolved an observed connection problem; its necessity for the final successful playback was not isolated in a separate control trial.

2.2 Independent DRM evidence

The Bitmovin DRM demo [7] reported EME/Widevine support but failed with `DRM_MEDIA_KEY_INITIALIZATION_FAILED`. A small DevTools test obtained Widevine access and then called `createMediaKeys()`; it returned `NotSupportedError: CreateCdmFunc not available`.

`chrome://histograms/Media.EME.Cdm` showed eight CDM-load samples in bucket 2. Chromium's enum defines the third, zero-indexed value as a library load failure, distinct from a missing file or missing entry point [3]. This interpretation used the upstream source checked during the session, not a symbolized trace from the installed release.

Check	Result before patch
CDM file and expected exports	Present
Module architecture and signature	arm64; Google signature valid
Loading through Python outside the browser	Succeeded
Browser CDM registration	Enabled
Browser MediaKeys initialization	Failed

An alternate Google-distributed Widevine 4.10.3050.0 package was acquired through the Firefox manifest [6]. Its 20,189,918-byte download and SHA-512 matched that manifest. After replacing the module and restarting, the independent DRM demo failed again. The newer copy was restored. Merely changing CDM versions did not solve the observed failure.

3. Working intervention

3.1 Why signing was investigated

Inspection of the original general Helium helper showed hardened runtime and library validation, with no disabling entitlement. Its signing team was input LLC (S4Q33XPHB4), whereas Widevine was signed by Google (EQHXZ8M8AV). Apple's documentation explains that ordinary library validation restricts loading to Apple-signed code or code signed by the same team; an exception permits third-party modules [4]. This made the observed signing mismatch a plausible explanation for the browser-only load failure.

No definitive dynamic-linker rejection message was recovered. The conclusion is therefore an inference supported by configuration and intervention results, not a directly logged rejection. Neon was inspected as a related prior implementation [8]; its installer was not run. The locally implemented patch retained hardened runtime rather than reproducing Neon's broad re-signing command unchanged.

3.2 Exact changes made by the final patch

The complete app was copied to a staging directory. Widevine was added under the active Helium framework version at `Libraries/WidevineCdm`. Each helper app, the framework, and the outer app were re-signed with an ad-hoc local signature, in that order. Existing ordinary entitlements were read as XML and retained, including the renderer's JIT entitlement.

The exception `com.apple.security.cs.disable-library-validation = true` was added to the outer app and every helper app. It was not limited to the CDM helper. Three vendor-bound keys were removed where present: `com.apple.application-identifier`, `keychain-access-groups`, and `com.apple.developer.web-browser.public-key-credential`. The framework was also re-signed but was not given the library-validation exception. This replaced the vendor signing identity; it did not create a new vendor-authorized build.

The signing command specified `--options runtime`. No `--no-sandbox` option was used. Observed child-process arguments included seatbelt sandbox configuration. These observations show that the procedure did not intentionally disable Chromium's sandbox; they are not a complete security assessment.

3.3 Isolation, mistakes, and corrections

The first patched copy used a separate empty profile. The protected demo played, and a private window in that copy played Crunchyroll after the user signed in. The episode advanced from 11:07 to 11:20. A subsequent clean-profile test used normal Keychain access and protected demo playback reached 01:09. The final main installation also used normal Keychain access.

Two preparation errors were also corrected: `codesign` entitlement output required `--xml` to preserve entitlements correctly, and Finder metadata in the Documents staging copy interfered with signing. Staging outside that file-provider location and removing only `FinderInfo/ResourceFork` metadata resolved the signing problem. Temporarily disabling component updates also disabled registration in the empty test profile; that test flag was removed.

4. Repeating the procedure on an Apple silicon Mac

This protocol reproduces the tested workflow with the supplied scripts. It is not an unattended installer or a cross-platform recipe. Read section 6 before applying it to an everyday profile. The scripts require Python 3, macOS signing tools, sufficient backup space, an official Helium installation, and an arm64 Google-signed Widevine module. About 1.9 GB of user data and 336 MB of application data were present in this case; required space varies.

Step 1: record the failure before changing anything

Record Helium's version from `chrome://version`, the player error, and the target episode. Check `chrome://components`, `chrome://media-internals`, the browser console, and the independent DRM demo [7]. A component being listed is not a passing playback test. If the problem is a blocked hostname, verify that separately; do not assume every KAT-6005 has the cause found here.

Step 2: establish a verified Widevine source

Keep `copy_widevine_from_chrome.py` and `repair_helium.py` together with the provided command files. On a fresh machine, install Chrome from Google's official source, then run:

```
python3 copy_widevine_from_chrome.py
```

The helper verifies Chrome's signature and Google team identity, locates its bundled module, verifies the arm64 library, and copies the whole module into `~/Library/Application Support/Helium DRM Repair/Verified Widevine/<version>`. It does not redistribute or download a CDM. If Chrome's package layout changes, it stops rather than guessing. This machine already has that verified cache and does not need this acquisition step again.

Step 3: prepare and inspect the replacement

With an official, unmodified Helium source installed, run:

```
python3 repair_helium.py prepare
```

Preparation verifies the official Helium signature and expected team, verifies Widevine's Google signature, stages and signs the replacement, and performs deep signature verification. It does not replace the installed app. The tool stops if Helium already bundles Widevine, so an official solution can be reassessed first. An already patched installation is rejected as a preparation source.

Step 4: back up and apply

Save open work and quit the main Helium. Then run:

```
python3 repair_helium.py apply
```

Application creates a full closed-profile backup and an original-app backup before replacing the installed app. It preserves the main profile rather than copying test-profile credentials into it. The script refuses to proceed while the normal main process is running and rechecks the staged signature. Use the supplied code, not just a bare `codesign --deep -s -` command: preserving entitlements and retaining a rollback copy are material parts of this procedure.

5. Verification, updates, and recovery

Step 5: launch and approve the correct Keychain request

Open /Applications/Helium.app normally, without a mock-keychain argument. If the expected Helium Safe Storage prompt appears, review it and allow access. The user handled this prompt directly in the investigation. Do not delete the Safe Storage item or reset the Keychain as a shortcut. Apple's description of the Allow/Deny choices provides the relevant permission behavior [5]. Approval can be requested again after re-signing or replacement; persistence was not tested across future releases.

Step 6: require actual protected playback

Open the independent DRM demo, start the video, and verify advancing time without a MediaKeys error. Then test a Crunchyroll episode available to the signed-in account. Check for a visible video frame, the player's advancing timer, and an audio-playing indication. Registration alone, a static thumbnail, or a successful HTTP request is insufficient. Pause after verification to avoid unnecessarily advancing viewing progress.

Trial	Measured outcome
Original app + Widevine 4.10.3112.0	Registration succeeded; DRM initialization failed
Secure DNS changed to Cloudflare	Endpoint connection succeeded; Crunchyroll still failed
Original app + Widevine 4.10.3050.0	Independent DRM initialization still failed
Patched isolated copy	Protected demo played; Crunchyroll advanced 11:07 to 11:20
Patched copy, normal Keychain allowed	Protected demo played to 01:09
Patched main, normal Keychain allowed	Existing login retained; video/audio status; 11:20 to 11:28

What happens after an update?

An official app replacement should be expected to remove local signing changes and the bundled addition. This expectation is consistent with Neon's design to reapply patches after browser updates [8], but a future Helium update was not tested here. No background patcher or update-disabling mechanism was installed.

If an update removes playback, quit Helium and run **Repair Helium.command**, then type REPAIR when asked. It runs preparation and application against the new official installation. If the updater rejects a modified app, reinstall the current official Helium release without deleting the profile, then run the repair tool. A new release may require changes to the procedure; signature checks passing do not prove playback compatibility.

During report preparation, the profile's Widevine copy was found absent after bundled-module use. A verified local cache was added to the repair tool. Cache-only preparation was then tested against the preserved official app in a disposable staging directory; it produced a replacement that passed deep signature verification. That test did not install another patch or validate a future release.

Undo and backup handling

Quit Helium and run **Restore Helium.command**, typing RESTORE. It restores the app from the latest repair backup while leaving current browsing data in place. If that backup predates a security update, prefer reinstalling the current official release rather than restoring an old app. A profile rollback is a separate, manual recovery action; the restore command does not overwrite newer browsing data. Backups may contain cookies and credentials, so keep them private. They are retained locally and accumulate across repairs.

6. Security tradeoffs and limits of the findings

Follow-up correction: X failure attributed to the internet connection. The signed-in X page initially loaded without retrieving timeline and account data. After further investigation, the user reported that their internet connection caused the problem. This supersedes the earlier unresolved-regression wording: the X incident is not treated as evidence that the DRM patch broke normal browsing. The network diagnosis is user-reported; an independent controlled network test was not performed as part of this report. This correction does not establish compatibility with every website or browser feature.

The library-signing protection is deliberately reduced. The patch permits third-party libraries in the app and helpers. Retaining hardened runtime and the browser sandbox does not restore the removed validation rule. The user explicitly approved this tradeoff before use in the main profile. The patch is not equivalent to an official, vendor-signed DRM integration.

Ordinary Keychain access was demonstrated, not every credential feature. After approval, the main Crunchyroll session remained usable without the test-only flag. Saved-password autofill, passkeys, enterprise device-trust keys, and hardware-backed credential functions were not tested. Removing vendor-bound entitlements may affect these features. A working streaming session must not be presented as proof of universal credential compatibility.

The causal evidence is strong but incomplete. A valid Google module loaded outside Helium, failed to initialize inside the official browser, and worked after a combined signing-and-placement patch. The team mismatch and Apple's documented validation rule support the library-validation hypothesis. The study did not isolate the entitlement change from module placement, produce a definitive dyld denial, or run a controlled reverted-patch trial. The initial browser update is another confounding change.

The scope is one machine and one episode. Intel Macs, Windows, Linux, other macOS versions, Netflix, Disney+, different quality levels, full episode completion, and future Helium or Widevine versions are outside the verified result. The test was a troubleshooting case study, not a statistically designed performance experiment or an independent security audit.

Integrity verification has a limited meaning. Successful `codesign --verify --deep --strict` checks confirm the integrity of the signatures present. After patching, those are local ad-hoc signatures, not imput's original identity. The repair script validates expected vendor team IDs as a guard against an arbitrary source, but its preparation/application change check hashes only the main executable; it is not a complete immutable-tree comparison. Apply promptly after preparation and stop if the installation changes.

7. Conclusion

The investigation restored the tested Crunchyroll episode in the user's main Helium installation, with its existing account session and normal Keychain access. DNS configuration and DRM initialization were distinct problems. The remaining workaround is a locally re-signed browser with a library-validation exception, supported by app/profile backups and an explicit repair process after updates. The result demonstrates local Crunchyroll playback compatibility. The later X loading problem was attributed by the user to their internet connection, rather than the DRM workaround. Official support, full credential compatibility, and future-update reliability remain outside the verified result.

Supplementary materials

The accompanying files are `repair_helium.py`, `copy_widevine_from_chrome.py`, `Repair_Helium.command`, and `Restore_Helium.command`. Keep the command files beside the Python repair script. The command wrappers use the Python installation present on the investigated Mac; other machines can run the explicit Python commands in section 4. No Widevine binary, browsing profile, authentication token, or private account information is included in the paper's supplementary files.

Appendix A. Read-only diagnostic commands

These commands inspect a live installation. For a patched machine, inspect the preserved original app to reproduce the original signing evidence. Replace the paths as necessary; Versions/Current is the framework's active-version symlink.

```
codesign -dv --verbose=4 /Applications/Helium.app
codesign --verify --deep --strict /Applications/Helium.app

HELIUM_FW='/Applications/Helium.app/Contents/Frameworks/Helium Framework.framework'
codesign -d --xml --entitlements - \
"$HELIUM_FW/Versions/Current/Helpers/Helium Helper.app"
```

The DevTools initialization test used during diagnosis was:

```
navigator.requestMediaKeySystemAccess('com.widevine.alpha', [{
  initDataTypes: ['cenc'],
  videoCapabilities: [{
    contentType: 'video/mp4; codecs="avc1.42E01E"'
  }]
}])
.then(access => access.createMediaKeys())
.then(() => console.log('CDM OK'))
.catch(error => console.error(error.name, error.message));
```

This distinguishes a failed MediaKeys initialization from a simple capability declaration. CDM OK is still not proof of license acquisition or playable media. Finish with the actual protected-video test.

Appendix B. Integrity identifiers

Values below identify this case's artifacts; they are not a universal allowlist for later versions. Hashes were collected during report preparation on October 2, 2026.

Artifact	Identifier
Official Helium team	S4Q33XPHB4
Google Widevine team	EQHXZ8M8AV
Original Helium main executable SHA-256	19903ede784a579f816d212256d0cd6dc727a817ef4a1087f814856df1484805
Widevine 4.10.3112.0 arm64 library SHA-256	e1e9f6b0c77f4251542a2f49dda419c85eda88c5d9ef2721d5d594dc195a6e5f

The original-app/profile backup resides locally beneath ~/Library/Application Support/Helium DRM Repair/Backup 20261002-130842. Do not distribute that directory with the report. The report intentionally omits the user's name, email address, and unrelated browser history.

References

External references were checked October 2, 2026. Repository links to moving branches may change. The local measurements reported above come from the tool outputs and visible browser state in this troubleshooting session; a complete raw trace is not attached.

- [1] imput. [Helium issue 116: DRM playback support](#). Public feature request; contextual evidence, not a diagnosis of this particular Mac.
- [2] imput. [Helium pull request 822: allow WidevineCdm installer](#). Reference for component installation support.
- [3] Chromium Authors. [CDM load-result enumeration](#). Used to interpret load-result bucket 2.
- [4] Apple. [Disable Library Validation Entitlement](#). Library-validation rule and exception.
- [5] Apple. [Allow apps to access your Keychain](#). Meaning of the Keychain permission choices.
- [6] Mozilla. [Firefox Widevine distribution manifest](#). Source URL, file size, and SHA-512 for the alternate Google package tested.
- [7] Bitmovin. [DRM Stream Test](#). Independent protected-media test used during diagnosis and validation.
- [8] bfayers. [Neon DRM patch implementation](#) and [project description](#). Prior implementation inspected for comparison; not installed or executed.